Provenance

Provenance I

Provenance records *how the release came to be*. The descriptor's `provenance` list is an ordered chain of steps, each naming the agent responsible. fig. 1 renders that chain via `provenance_steps()` in `src/data_descriptor/figures.py`.

Provenance I

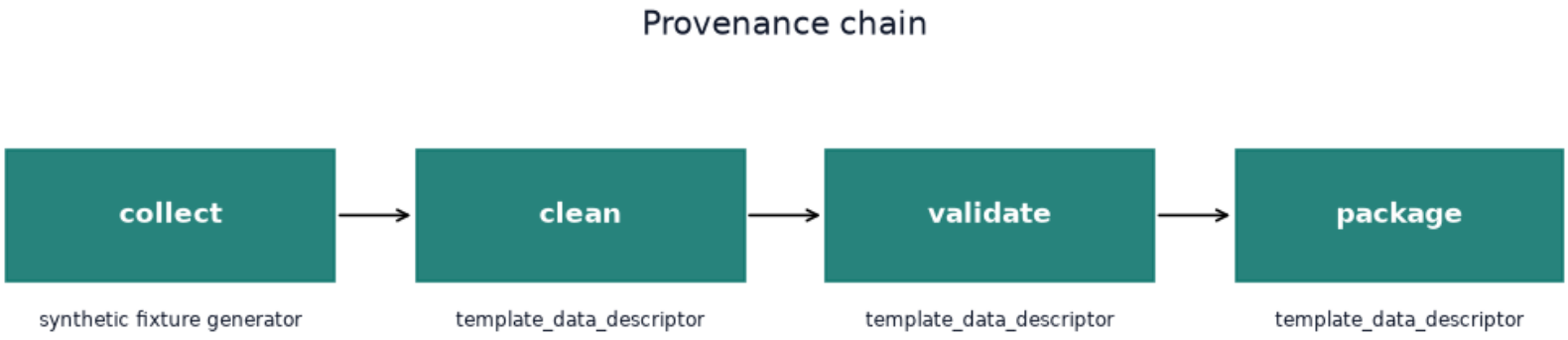

Figure 1: Provenance chain: the ordered steps that produced the release, from `provenance_steps()`. Each box is a declared step; the label beneath is the responsible agent. The chain runs left to right from raw collection to the packaged, metadata-only release manifest.

Methods: the provenance chain I

The provenance chain is this descriptor's methods section — it records the generation and processing methodology step by step. The shipped descriptor declares four steps:

Methods: the provenance chain I

1. **collect** — a synthetic fixture generator emits the deterministic measurement and subject tables.
2. **clean** — identifiers are normalized and the measurement range is bounded.
3. **validate** — the descriptor is checked for schema, constraint, and byte-level agreement.
4. **package** — the metadata-only release manifest is emitted.

Why depth matters I

The validator treats provenance shorter than two steps as a warning — a release that records only "we have the data" but not "how it was produced and checked" is thinner than a reusable descriptor should be. The four-step chain here pairs a collection origin with an explicit validation and packaging trail, which is the minimum a downstream reuser needs to judge whether the dataset was produced the way they require. For a real dataset, these steps would cite concrete tools, versions, and operators rather than the template's synthetic agents.